



CityBites

Team 3

Suppapoo 6681437

Natthakul 6680972

Pakawun 6681453

Chawinthorn 6681402

Matteo 6480996

Nattapas 6681399

Project Overview & The Problem



- **The Problem:** Orders are currently scattered across phone calls and chat apps.
- **Pain Points:**
 - **Staff:** Disorganized and overwhelming communication.
 - **Customers:** Zero visibility into their order status.
 - **Owner:** Increased operational costs due to small, frequent order mistakes.
- **Why it Matters:** Fragmented workflows create bottlenecks and cause operations to break down during busy hours.

Customer

A person who orders the items and sees the list of available items on the menu along with the price, they interact with the real time tracker

Staff

A CityBite staff member who manages incoming orders, confirms details, and updates the status of food preparation. Staff are currently overwhelmed by scattered communication across phone and chat; they need one centralized place to verify exact order contents and customer requests

Owner

The business owner who wants to reduce operational costs slips. They care about having a clear overview of operations and ensuring the workflow remains stable even when the shop gets busy.



Target Users

- **Centralization:** The orders now will be recorded in one page or one dashboard so the orders will not scatter across the chat and phone call which will help staff to handle it better.
- **Menu accuracy:** Staff only update the menu in one place for it to show to all customers.
- **Order restrictions:** The unavailable items will not be submitted as the system prevents it.

- **Inquiry load:** Reduction in phone call and message just to ask the question "what when why how".
- **Structured List:** Moving all informal chat and phone call data to structured order list form, reducing the misunderstanding.
- **Status visibility:** Offering the place to check for the status of the order prevents a ton of questions.

- **Customer feeling:** They get an immediate response on what is available, improving their experience.
- **Immediate use:** The business can actually use it immediately without a large IT team to manage it.



Success Criteria

Project Scope & Key User Stories

Core

- Registration and login
- Menu browsing for everyone.
- Dish customization (e.g., "no spice") for customer.
- Centralized order inbox for staff.
- Menu availability toggling for staff.
- Real-time order status for customers.

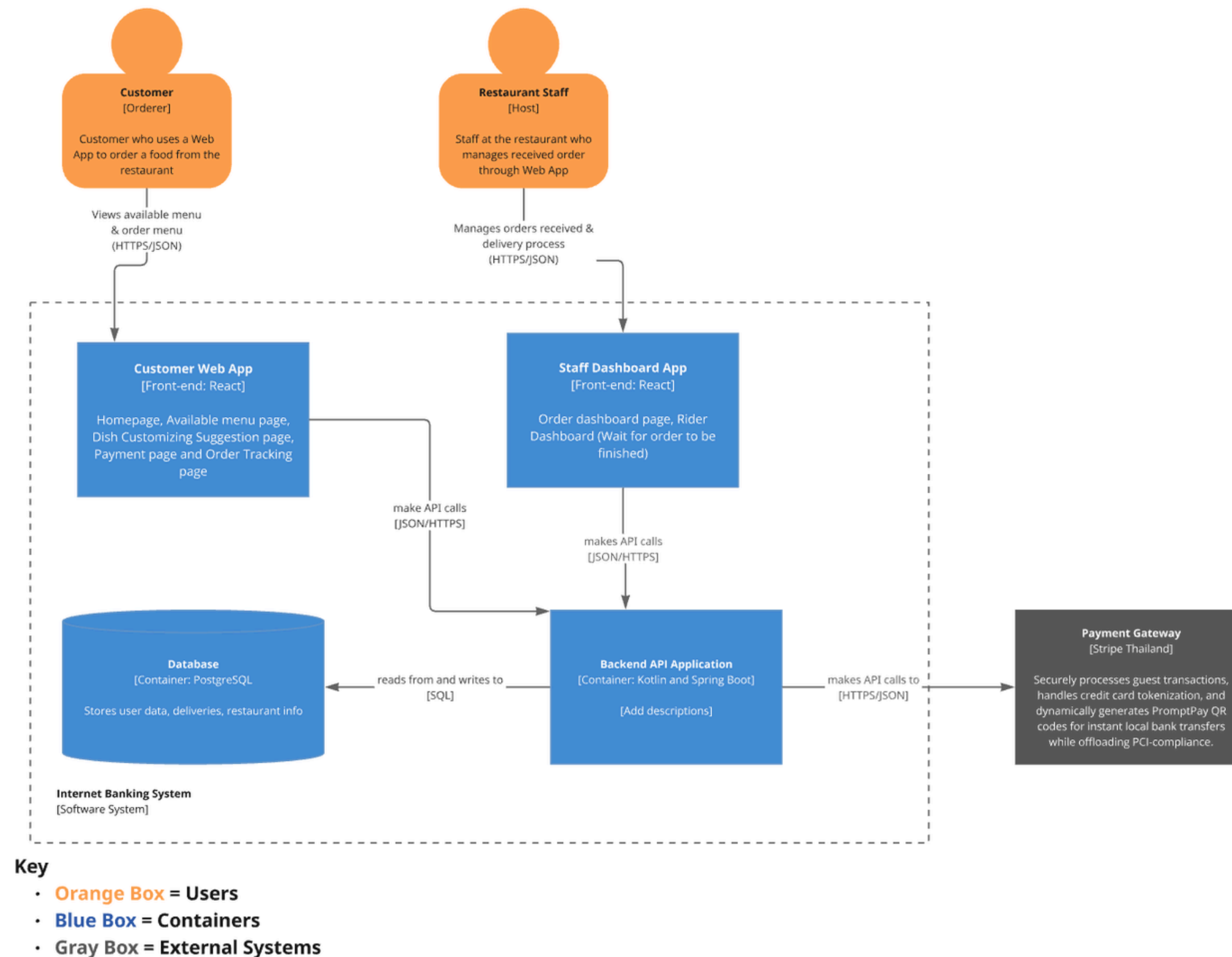
Out-scope/Optional

- ETA GPS tracker (Estimated-time-arrival) for customer and staff

Immediate usability without requiring a large IT team



C4 Architecture



Data Storage Choice

PostgreSQL

- Perfect for mapping complex Menu-to-Order relationships.
- Handles high-concurrency order volumes during peak hours

External Services

Stripe & PromptPay

- Uses HTTPS/JSON for card tokenization and gateway communication.
- Generates PromptPay QR codes for instant Thai bank transfers.
- Real-time confirmation of payment to trigger kitchen workflows.

Main Request Flow

Order

Customer sends JSON payload to Backend via Web App.

Process

Backend validates stock and creates DB entry.

Pay

Gateway generates PromptPay QR for transaction.

Notify

Staff Dashboard reflects "Ready to Prep" status.

Progression & Sprint 1 Focus



Initialize Repositories & Database Schema

Frontend and backend repos initialized. Configuring the PostgreSQL database stack and currently mapping out base data schemas.

Completed



Setup Dockerfile Environments

Ports are mapped for isolation. Standardized **Dockerfiles** are written for both ends to compile, package, and containerize our codebase.

Completed



Multi-Container Orchestration (Docker Compose)

Integrated **docker-compose up** to spin up the database, frontend, and backend with one step, eliminating redundant manual commands.

Completed



Configure NGINX Reverse Proxy

Currently setting up NGINX to listen on port 80, acting as a single gateway that routes traffic to correct container destinations

In Progress

Sprint 2 Plan & Responsibility Split



Responsibilities

- **Nattapas:** ERD, Short-drafting database table
- **Chawinthorn:** ERD, authentication system
- **Suppapoo:** ERD, Short-drafting database table
- **Matteo:** Designing User Interface and other documents
- **Natthakul:** Designing User Interface and other documents
- **Pakawun:** Designing User Interface and other documents



Process

Our team holds weekly sprint meetings to review the backlog, address blockers, and ensure continuous integration so we can guarantee timely product delivery.

Risks, Assumptions & Next Steps

RISKS

Synchronization
during high loads

Potential Network Issues

Race Conditions

SOLUTIONS

Testing database heavily
Postgres is relational

- Retry requests
- Graceful error handling
- Temporary offline states
- Persistent order storage

Validate item availability server-side
at order submission, not just at
menu load time.

Risks, Assumptions & Next Steps

Assumptions

The expected number of concurrent users will remain within manageable limits for a single PostgreSQL instance.

Stripe/PromptPay APIs are stable and operational during transactions.

Staff update item availability correctly

Next Steps

- Finalize repository setups and push the initial database schema.
- Begin developing the "Customer browses categorized menu" feature for the Sprint 1 Demo.

Q&A



Please let us know if you have any questions or concerns that have not yet been addressed.



Thank *You*

